



Julia Ferreira Nery

Turno: Matutino | Turma: 102653

DESAFIO | Inglês Técnico e Lógica de Programação

O desafio busca a criação de um código para a obtenção de x valores, inseridos por um usuário específico, e, ao final, apresentar os cálculos exigidos para ele com base nos valores digitados.

No entanto, ao realizar a criação do código, um conflito foi percebido. Por mais que as informações pedidas fossem em valores numéricos, a possibilidade de que o usuário pudesse respondê-las com uma string (texto) ainda era existente e poderia acarretar em erro nos dados finais apresentados ao usuário. Ex.:

O consumo necessário é NaN litros
O menor valor pesquisado é R\$ Infinity
A média dos valores pesquisados é R\$ NaN
O gasto diário (ida e volta) é R\$ NaN

Então para evitar esse erro, um laço de repetição foi necessário. E foi com o uso do *while* que uma solução foi encontrada. Através dele foi possível criar um loop de segurança que se repete até que a validação seja dada como falsa. Ex.:

```
let distancia_Percorrida = parseFloat(prompt("Qual a distância percorrida da sua casa até seu trabalho (em km)?"));
while (Number.isNaN(distancia_Percorrida) || distancia_Percorrida <= 0) {
    alert("Por favor, digite apenas números válidos!");
    distancia_Percorrida = parseFloat(prompt("Qual a distância percorrida da sua casa até seu trabalho (em km)?"));
}
```

Nesse caso, a validação está sendo feita com uma das variáveis do código, chamada: *distancia_Percorrida*. Com o *while* o valor que for inserido nessa variável entra em um loop de validação onde determina se o que foi inserido pelo usuário é NaN (Not a number), caso a sentença seja verdadeira, o código entra em repetição até que o valor inserido seja um valor numérico válido. E quando a sentença for dada como falsa, a execução do código dá continuidade. E esse mesmo loop é feito com as outras variáveis que exijam um valor numérico.

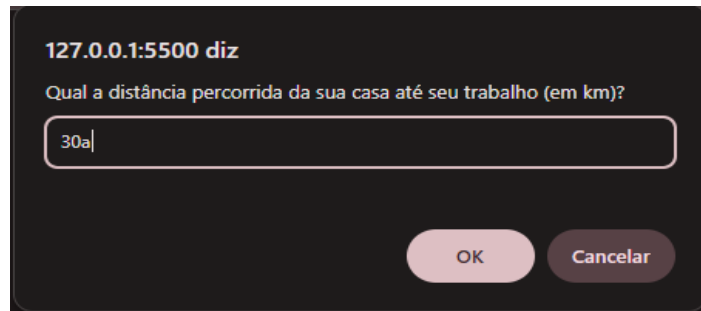
E dentro do *while* usamos 2 situações de validação.

- *Number.isNaN()*: Uma função para determinar se o valor é NaN de forma precisa
- *<= 0*: Refere-se que se o valor da variável anterior a ele for menor ou igual a 0, ele entra dentro do loop

Ambas as situações contribuem para que o usuário insira, exatamente, os valores exigidos pelo programa para a determinação do cálculo final.

Diante disso, uma questão curiosa é: Por que usar a função *Number.isNaN()* ao invés da função global *isNaN()*? No site [Stack Overflow](#), temos um comentário com a seguinte pergunta: “How do you test for NaN in JavaScript?”. E aqui nós podemos perceber que a função *isNaN()* apresenta uma certa irregularidade na sua validação de se o valor é NaN ou não. Portanto, sua troca por *Number.isNaN()* é mais segura pois ela é mais rigorosa na sua confirmação em NaN.

Porém, mesmo com todas essas validações pode ocorrer uma situação parecida com:



Tendo isso em vista, para que a string (a) não passe junto com o número e o resultado seja NaN é preciso o uso da função *parseFloat()*. De acordo com o [3WSchool](#), o *parseFloat()* é um método de conversão de string para um número flutuante (decimal). E ele funciona da seguinte forma:

- I. Lê da esquerda para a direita;
- II. Se o primeiro caractere da string não for um número, um sinal (+ ou -), um ponto (.) ou espaçamento, ele desiste de continuar a leitura e retorna NaN;
- III. Ele extrai até onde consegue identificar como número.